A firmware APL time-sharing system

by RODNAY ZAKS,* DAVID STEINGART,* and JEFFREY MOORE**

University of California
Berkely, California

INTRODUCTION

Incremental advances in technological design often result in qualitative advances in utilization of technology. The recent introduction of low-cost, microprogrammed computers makes it feasible to dedicate highly sophisticated and powerful computation systems where previously the needed performance could not be economically justified. Historically, the contribution made by the computing sciences to the behavioral sciences has been limited largely to statistical analysis precisely because sufficiently sophisticated computing equipment was available only outside the experimental situation. Inexpensive time-sharing systems have recently made it possible to integrate the computer in a novel way as a tool for conducting experiments to measure human behavior in laboratory situations. A detailed presentation of computerized control of social science experimentation is presented later. However, many aspects of the system are of general interest because they exploit the possibilities of a newly available computer generation.

Iverson's APL language has been found to be very effective in complex decision-making simulations, and the source language for the interpreter to be described is a home-grown dialect of APL. It is in the nature of interpreters that relatively complex operations are performed by single operators, thus making the ratio of primitive executions to operand fetches higher than in any other mode of computation. This is especially true in APL, where most bookkeeping is internal to particular operators, and a single APL operator may replace a FOR block in ALGOL, for example. This high ratio places a premium on the ability of microprogrammed processors to execute highly complex instruction sequences drawing instructions from a very fast control memory instead of from much slower core memory.

In the new generation of microprogrammable computers the microinstructions are powerful enough and the control memories large and fast enough to permit an on-line interpreter and monitor to be implemented in microcode. If a sufficient number of fast hardware registers is available, core memory serves only for storage of source strings and operands. The speed advantages of such a mode of execution are obvious.

SYSTEM ARCHITECTURE***

META-APL is a multi-processor system for APL time-sharing. One processor ("the language processor") is microprogrammed to interpret APL statements and provide some monitor functions, while a second one (the "Interface processor") handles all input-output, scheduling and provides preprocessing capabilities: formatting, file editing, conversion from external APL to internal APL. Editing capabilities are also provided offline by the display stations. In the language processor's control memory reside the APL interpreter and the executive. In the Interface processor's reside the monitor and the translator.

An APL statement is thus typed and edited at a display station, then shipped to the Interface processor which translates and normalizes this external APL string to "internal APL," a string of tokens whose left part is a descriptor and right part an alphanumeric value or i.d. number corresponding to an entry in the user tables (see appendix A). External APL may be reconstructed directly from internal APL and internal tables, so that the external string is discarded and only its internal form is stored. This internal APL string is shipped to the language processor's memory: the APL processor will now execute this string at the user's request.

---

* Department of Computer Science
** School of Business Administration

*** The concepts presented here are being implemented under the auspices of the Center for Research in Management Science. Research on the final configuration is continuing at the Center and the views expressed in this paper represent the ideas of the authors.

From the collection of the Computer History Museum (www.computerhistory.org)

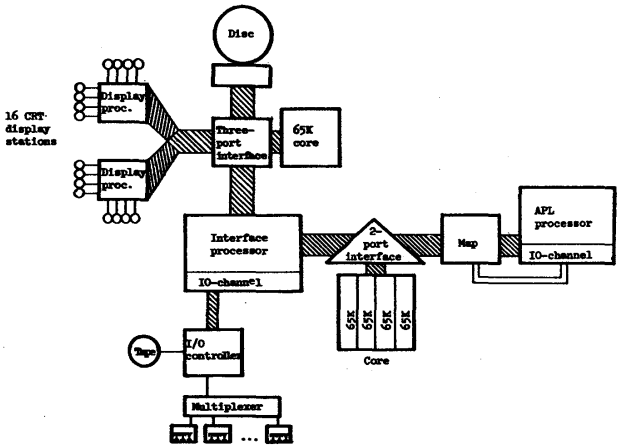


Figure 1—The Meta-APL time-sharing system (projected)

The variable's i.d. numbers ("OC#") are assigned by the system on a first-come-first-served basis and are used for all further internal references to the variable. This OC# is the index which will be used from now on to reference the variable within the OAT (Operand Address Table) of the language processor. The set of internal APL strings constitutes the "program strings."

Microprogramming encourages complex interpretation because the time spent interpreting a given bit or string of bits is negligible. We have taken advantage of this ability to allow short descriptors to replace "dead-data" wherever possible so as to minimize the inert-data flow and maximize the active-data flow. All external program symbols are translated to tokens internally—however, as we have previously mentioned, the grammar and semantics of the internal notation are isomorphic to the external symbolic.

APL PROCESSOR: HARDWARE

The laboratory requirements called for a very fast APL processor capable of executing at least sixteen independent or intercommunicating experimental programs, each program responding in real time to textual and analog input from the subject of the experiment.

Once the possibilities of a microprogrammed interpreter became apparent, the search for a machine centered on those with extensive microprogramming facilities. Of these the Digital Scientific Meta-4 was chosen by the Center for Research in Management Science for its fast instruction cycle, extensive register complement, and capable instruction set.

The processor fetches 32-bit instruction from a read-only memory on an average of every 90 nsec. Instructions fetch operands from thirty-one 16-bit-registers through two buses and deposit results through a third into any register. Most instructions may thus address three registers independently—there are no accumulators as such. Up to 65K of 750 nsec cycle core may be accessed through two of the registers on the buses, IO through another pair, and sixty-four words of 40 nsec scratch pad memory through yet another pair. These registers are addressed as any others and the external communications are initiated by appropriate bits present in all microinstructions.

Triple addressing and a well-rationalized register structure promote compact coding. The entire APL interpreter and executive reside in under 2,000 words of read-only memory.

Although special multiply and divide step micro-instructions are implemented in the hardware of the Meta-4, the arithmetic capability of the processor is not on a par with the parsing, stack management, and other nonarithmetic capabilities of the interpreter. Adding a pair of 32-bit floating operands takes about 5 μsec, a very respectable figure for a processor of this size and more than adequate for the laboratory environment. A floating multiply or divide takes 20-25 μsec.

On the other hand, a pass through the decision tree of the parser takes 1-2 μsec, and as will be seen from the descriptor codes this tree is fairly deep. This speed is a consequence of the facility to test any bit or byte in any register and execute a branch in 120 nsec, or mask a register in less than 90 nsec.

APL PROCESSOR: MEMORY

The experimental situation demands that response time of the computer system to external communication be imperceptible. We were forced by this consideration to make all active programs resident in core, and

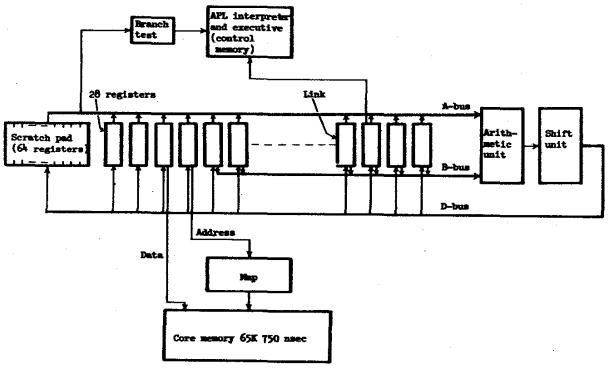


Figure 2—The APL processor

From the collection of the Computer History Museum (www.computerhistory.org)

in order to maximize the utilization of the available address space of 65K, several designs evolved.

1. Through a hardware map, the virtual address space of 65K is mapped into 256K core.
2. Since many of the APL operators are implemented in core, and since the experimental situation normally requires many identical environments (programs with respect to the computer), all program strings are accessible concurrently to all processes or users.
3. Through dynamic mapping of the available physical memory space, individual processes may be allocated pages as needed, and pages are released to free space when no longer occupied by a process. Optimal use is made of the waxing and waning of individual processes.

The entire virtual memory space is divided into three contiguous areas: system tables; system and user program strings, processes work space. Within the processes work space, memory is allocated to the stack and management table (MT) for each process. The stack and MT start in separate pages, the stack from the bottom and the MT from the top, and these two are mapped as the bottom and top pages of the virtual work space, regardless of their physical location. As the process grows during execution, pages are allocated to it from free space within the process work space and are mapped as contiguous memory to the existing stack and MT.

The specifics of the memory and map design were constrained primarily by available hardware. The computer used has a 16-bit address field—65K is the maximum direct address space but not adequate for 40 plus processes. Mapping by hardware 256K into 65K eliminates the need for carrying two-word addresses inside the computer. Pages are 512 words long, 128 pages in the 65K virtual space, 512 pages in the real space, keeping fragmentation to a minimum.

The map is a hardware unit built integrally with the memory interface. The core cycle is 750 nsec, the map adds 80-100 nsec to this time.

The map incorporates a 128-word, 12-bit, 40 nsec random access memory which is loaded every time a user is activated. The data comprising the user page map are obtained from a linear scan of the general system memory map.

Each word in the map contains three fields.

- In the n-th word in the hardware map, the right-hand seven bits contain the physical address of the page whose virtual address is n.

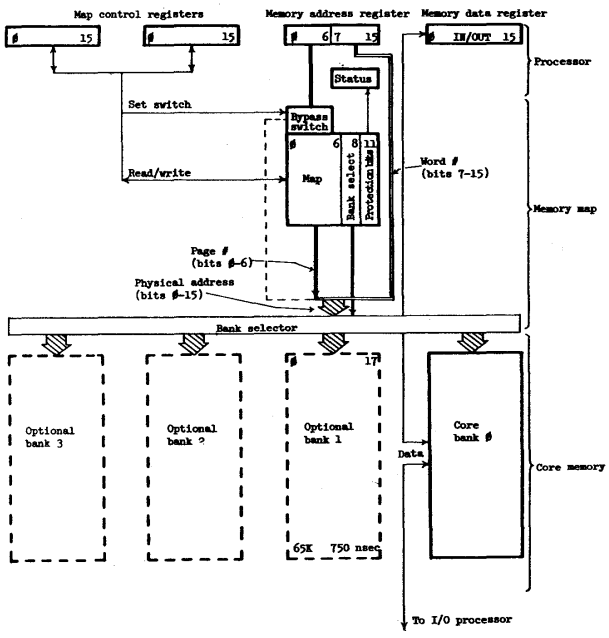


Figure 3—The hardware map

- The two bits adjacent to this field (bits 7, 8) map the 65K space into 256K (i.e., bank select).
- The three remaining have control functions and are returned in about 100 nsec to the status register associated with the memory address register. These bits are thus interpreted by the microprogram and any actions necessary are taken before the memory returns data 350 nsec later, 450 nsec after the initiation of the read (or write).
- The first bit, when set, causes an automatic write abort, thereby providing read-only protection of a given page.
- The second bit, when set, indicates a page fault. When detected in the status register, a special routine is executed which allocates a new page to the user.
- The third bit indicates that a page is under the control of the interface processor and prevents the APL processor from modifying or reading that page.

In general, the program string area is protected by the read-only bit; it may be modified only by the interface processor. All free storage in the processes work space, and all virtual pages not allocated to the process activated at a given time are protected by the page fault bit. Thus, when a process references outside its unprotected area, the request is interpreted as a request for additional storage. When the interface processor is

From the collection of the Computer History Museum (www.computerhistory.org)

modifying either program strings or input-output buffers, those areas are protected by the read-write abort bit.

The map may be bypassed by setting the appropriate bits in the map access register. This is to permit loading of the map to proceed simultaneously with core fetches, while the map's memory settles down, and to avoid the bootstrapping necessary if the map always intervened in the addressing of core.

The system memory map, stored in the top section of the user's virtual storage establishes the system-wide mapping from physical to virtual pages. Each of the 128 entries, one per physical block contains the owner's ID number (or zero) and the corresponding virtual location within its storage. Free pages are chained with terminal pointers in CPU registers. The overhead incurred in a page fault or page release is thus minimum (3 μsec).

APL PROCESSOR: INTERPRETER SOURCE

The APL interpreter accepts strings of tokens resident in the program strings area of core. The translation from symbolic source to token source is performed externally to the APL processor by an interface processor. The translation process is a one-pass assembly with fix-ups at the end of the pass for forward-defined symbols. The translation preserves the isomorphism between internal and external source and is *naturally* a bidirectional procedure—external source may be regenerated from internal source and a symbol table.

Meta-APL closely resembles standard APL with some restrictions and extensions. The only major restriction is that arrays may have at most two dimensions—a concession toward terseness of the indexing routines. There are two significant extensions.

Functions

Functions may have parameters, which are called by name, in addition to arguments. This is to facilitate the development of a "procedure library" akin to that available in ALGOL or FORTRAN. Parameters eliminate the naming problem inherent in shared code.

The BNF of a Meta-APL function call:

⟨FUNCTION CALL⟩:= {⟨ARGUMENT⟩}
⟨FUNCTION NAME⟩
{(⟨PARAMETER LIST⟩)}
{⟨ARGUMENT⟩}

⟨PARAMETER LIST⟩:= ⟨VARIABLE NAME⟩ |
⟨PARAMETER LIST⟩,
⟨VARIABLE NAME⟩

The variables specified as parameters may either be local to the calling function or global. The mechanics of the function call will be described later, as this is one of the aspects of this implementation which is particularly smooth.

Processes

The other significant extension in Meta-APL is the facility of one program to create and communicate with concurrently executing daughter programs, all of which are called processes. Briefly, a process is each executing sequence represented by a stack and management table in the "processes work space." Any process can create a subprocess and communicate with it through a parameter list, although only from global level to global level. The latter restriction is necessary because processes are asynchronous and the only level of a program guaranteed to exist and be accessible at any time is the global level (of both mother and daughter processes).

The activation of a new program,

$NUprog{P1, P2, P3, ..., Pn} PROGRAM NAME

establishes a communication mechanism, the "umbilical cord" between calling program A and called program AA. AA constitutes a new process and will run in parallel with all other processes of the system. The cord however, establishes a special relationship between AA and A:

—the cord may be severed by either A or AA, causing the death of the tree of processes (if any) whose root is AA.
—the parameter list of the $NUprog command establishes the communication channel for transmitting values between A and AA. All these parameters may thus be referenced by either process A or process AA and will cause the appropriate changes in the other process. To prevent critical races, two commands have been introduced.

$WA (WAITFOR) which dismisses the program until some condition holds true.

$CH (CHECK) which returns 1 if the variable has already been assigned a value by the program, ϕ otherwise. It expects a logical argument. Thus $WA ($CH V1 $\vee$ $CH V2) will hang program A until either V1 or V2 have been evaluated by program AA on the other side of the umbilical cord. It will then resume processing.

Among the parameters that may be passed are IO device descriptors. Hence, a mother process can tem-

From the collection of the Computer History Museum (www.computerhistory.org)

porarily assign to daughters any IO device assigned to her. This is to facilitate use of simple reentrant IO communications routines to control multi-terminal interactive experiments under the control of one mother process. The mother may identify daughters executing the same program string by assigning them distinct variables as parameters.

The usual characteristics of well ordering apply to process tree structures.

The BNF of Meta-APL is included as an appendix.

THE DESCRIPTOR MECHANISM

The formats of the internal tokens are as follows: numerical *scalar* quantities are all represented in floating point and fixed only for internal use. The left-most one bit identifies the two words of a floating operand: the 1-bit descriptor allows maximum data transit.

Operand calls

Variables are identified by descriptors called *operand calls* (after Burroughs). The i.d. field of the OC locates an entry in the Operand Address Table (OAT) which gives the current relative address of the variable in the stack, or specifies that a given variable is a parameter or undefined.

Another bit specifies the domain of the variable, local or global. Unlike ALGOL, there is one global block in Meta-APL. The possible addressing is indicated graphically.

When a process is created or a function is called, a block of storage is allocated to the Management Table to store the stack addresses of all the variables of that block—the block known as the Operand Address Table (OAT). The i.d. field of the OC is an index to the OAT. When an OC is encountered as the argument of an operator, the address of the variable is obtained in or through the OAT. If the variable is local to the current block and defined, the current stack address is found in the appropriate location of the local OAT. If the variable is global, as specified by the domain bit, the global level OAT is accessed. In either case, if the variable is undefined, a zero will be found in the OAT entry for that variable, and an error message will result. If the variable is a parameter, an operand call to either the calling or global level will be found in the OAT. In the case of a function, this OC points to either the calling block OAT or the global OAT and the address/zero/parameter OC will be found there. If the OC was found in the global OAT, it is a parameter from the mother process as described above.

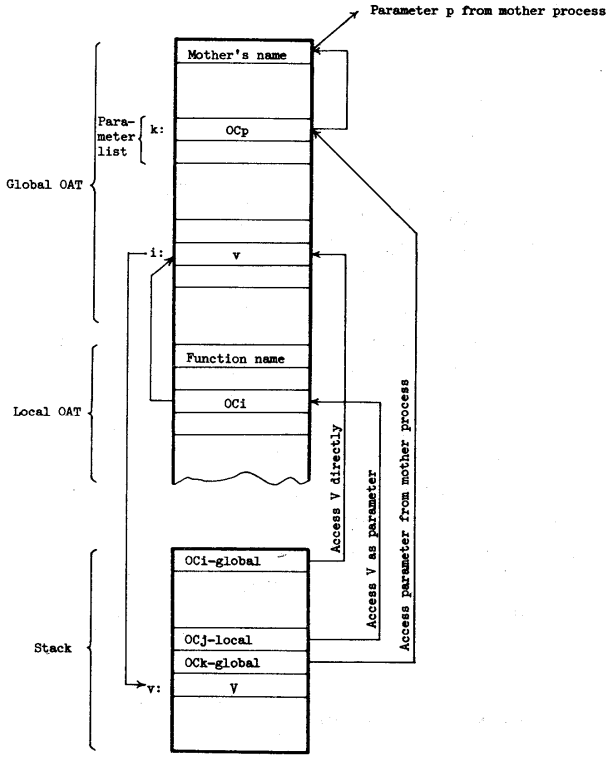


Figure 4—Stack addressing mechanism

Obviously, parameters may be linked through every level of process and function.

Operators

Operators are represented by such word descriptors containing tag bits, identification bits, and some redundant function bits specifying particular operations to the interpreter (marking the stack, initiating execution, terminating execution).

During parsing, operators are placed on an Operator Push Down List created for every block immediately below the OAT for that block. During the execution phase, operators are popped off the OPDL and decoded first for executive action (special bits) and number of arguments. The addresses of the actual operands are calculated as explained under *Variables* and those addresses passed to the operator front end. This routine analyzes the operands for conformability, moves them in some cases, and calls the operator routine to calculate results, either once for scalars, or many times as it indexes through vector operands.

From the collection of the Computer History Museum (www.computerhistory.org)

The operator front end represents most of the complexity of the execution phase since the variety of APL operands is so great.

Function call

The mechanism of *function call* uses the OPDL. If a function descriptor is encountered during the parse, it is pushed onto the OPDL and three zeroed words are left after it for storage of the dynamic history pointers. The specifications of the function are looked up in the function table and one additional zeroed word is left for each variable which appears in the function header before the function name, i.e., A←B FOO... would result in two spaces zeroed one for A, one for B.

Then, as the parse continues, if a left brace is encountered (as in A←B FOO{P_1, ..., P_n}C), parameter OCs P_1 through P_n are pushed onto the OPDL until the right brace is encountered. The number of parameters (n) is entered, the function descriptor duplicated, and parsing proceeds on its merry way.

During execution the last entered function descriptor is popped. This initiates the function call. First, the number n above is compared with the number of parameters specified in the function header. Then the address for arguments B and C are entered (these are the current top two elements of the stack). The current stack pointer, MT pointer, and program string pointer are saved in the appropriate locations and x words after the C argument are zeroed to accommodate the x new variables to be defined in the new function; (x was obtained from the function string header).

At this point, control is passed to the function program string with the new OAT already formatted.

The purpose of the preceding description is to indicate the kind of manipulation which is cheap in time and instructions in a microprogrammed interpreter. The function call routine takes under fifty instructions and takes about 10 μsec to execute (plus .9 x μsec to zero x locations).

Other descriptor types:

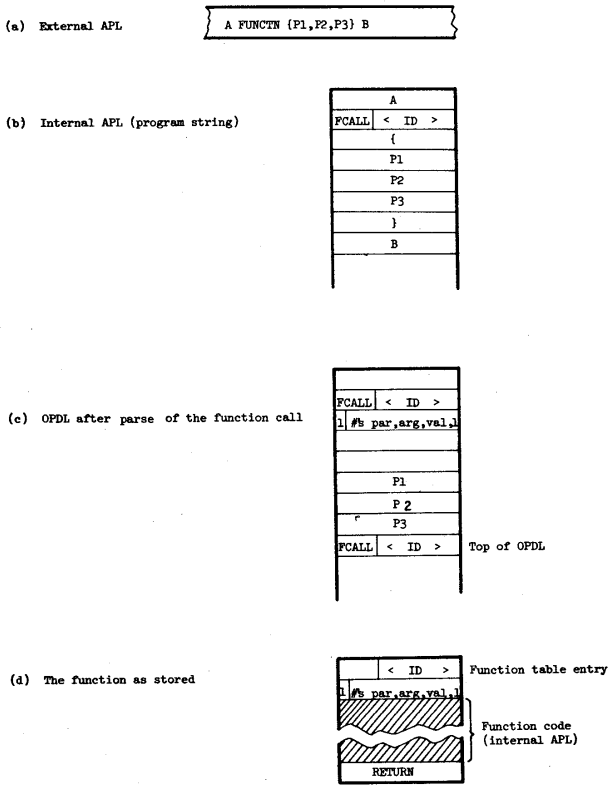


Figure 5—Function call

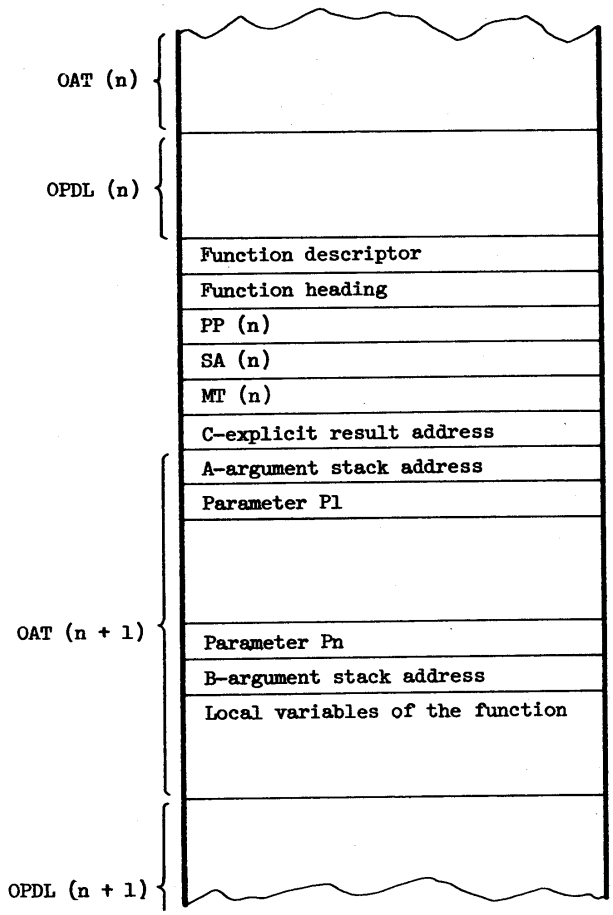


Figure 6—Management table after activation of C←A F00 (P1, P2, ..., Pn) B

From the collection of the Computer History Museum (www.computerhistory.org)

SYMBOL	#	BINARY																OCTAL	DESCRIPTION	
		Ø	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15			
A	Ø	Ø	X	P	O	N	E	N	T	<	.	M	.	.	A	.	.	Ø - - - - -	Arithmetic	Exponent=<1-7> 2's complement
		N	.	.	T	.	.	1	.	.	S	.	.	S	.	A	>	- - - - - -		Mantissa=<8-31> →first bit is sign(Ø=+,1=-)
OC	1-4	1	1	Ø	Ø	D	T	<	V	A	R	.	.	I	D	#	>	1 4 - - - -	Operand call	Domain: 1=global, Ø local Trace: 1=on, Ø=off Variable I.D. #=<6-15>
V	5-8	1	Ø	Ø	Ø	L		Y	.	.	L	E	N	G	T	H	>	1 Ø - - - -	Vector	Y length up to 11 bits
		<	X	.	.	L	.	E	.	N	.	G	.	T	.	H	>			X*Y must be < 16 bits
		<	N	U	M	B	E	R	O	F	.	W	O	R	D	S	>			followed by X elements and Y elements L=Ø=double words 1=half-words
PH	9	1	Ø	Ø	1	-	-	-	-	-	-	-	-	-	-	-	-	1 1 - - - -	Phantom	
OP	1Ø-11	1	Ø	1	Ø	T	R	M	D	E	<	O	P	E	R	A	>	1 2 - - - -	Operator	TR=trace M=mark stack]← D=dyadic=1, monadic=Ø E=execution delimiter:])⍋ Opera is the operator code(1 bit tells scalar operator)
SEG	42-45	1	Ø	1	1	D	D	-	-	-	-	-	-	-	-	-	-	1 3 - - - -	Segment-operator goes to OPDL	DD=ØØ=function call: bits 6-15=I.D.# Ø1=empty marker (NOOP for parser) 1Ø=program call 11=beginning of line + line number
IO	46-49	1	1	1	1	F	F	-	-	-	-	-	-	-	-	-	-	1 7 - - - -	IO descriptor goes to stack	FF=ØØ=output program# Ø1=input 1Ø=unused 11=unused
	5Ø	1	1	Ø	1	-	-	-	-	-	-	-	-	-	-	-	-	1 5 - - - -	Unused	
RET	51	1	1	1	Ø	-	-	-	-	-	-	-	-	-	-	-	-	1 6 - - - -	End of function definition=RETURN	

Figure 7—The descriptors

A TIME-SHARING SYSTEM FOR BEHAVIORAL SCIENCE EXPERIMENTATION

Historically, the contribution made by the computing sciences in the behavioral-science area has been limited almost exclusively to the utilization of computational resources for statistical analysis of social-science data and simulation studies. The advantages offered by the computer technology in other areas have until very recently been lost to the behavioral sciences. The advent of reliable and economical time-sharing systems has opened new vistas to the research horizons of a social-science experimenter. The use of time-sharing systems in programmed learning for teaching and other educational purposes has been well documented. The objective of this paper is to outline a science whereby the process-control technology combined with time-sharing can be used in a novel way as a tool for conducting experiments to measure human behavior in laboratory situations. Traditionally, attempts to monitor human behavior in decision-making situations have had less than desirable results, due primarily to the extreme difficulty in maintaining control over the course of the experiment. That is, subjects of a behavioral-science experiment often do not behave in a manner which is conducive to exercise of experimental control so that certain variables can be measured in a controlled environment. To meet the challenge of properly controlling experiments in the social sciences, a laboratory for that purpose was created by a grant from the National Science Foundation in the early 1960s.* The intent was to utilize computerized time-sharing with special-purpose hardware, combined with a suitable cubicle arrangement so that subjects participating in, for example, economic-gaming situations could have their decisions monitored and recorded by the time-shared computer. The idea was to have the experimenter model his economic game by writing his mathematical model as a computer program. At run-time, the resulting program serves as the executive controlling subject input from the time-sharing terminal. In this fashion, the computer serves two functions: to provide the medium whereby the experimenter may mathematically express his experimental design and to serve as the data-collection and process-control device during a time-shared experiment in which subjects at the terminals are communicating with the computer.

The requirements placed upon a time-sharing system

* A C HOGGATT J ESHERICK J T WHEELER "A Laboratory to Facilitate Computer Controlled Behavioral Experiments," *Administrative Science Quarterly*, Vol. 14, No. 2, June 1969.

From the collection of the Computer History Museum (www.computerhistory.org)

when it is utilized for computer-controlled experiments differ markedly from those placed upon a conventional time-sharing system. The actual implementation of the model itself requires a general-purpose computational capability combined with the usual string-handling capabilities found on any general-purpose time-sharing system; and hence, these are a minimal requirement of any experimental-control computer. The features which most notably distinguish a time-shared computer system when used for experiments are as follows: (1) Response of the processor to input from the remote terminals must be virtually instantaneous, that is, in experimental situations the usual delay of one or more seconds by the time-shared processor after user input is prohibitively long. In some measurement situations, in order not to introduce an additional and uncontrolled variation in his behavior, such as might be caused by even minor frustration with the responsiveness of the time-sharing system, feedback of a response to a subject's input on a time-shared terminal must be less than approximately five hundred milliseconds. In other less rigorous experimental situations in which rapid feedback is important, the relatively lengthy response time of most time-sharing systems has also introduced significant variation in subject behavior. (2) The measurement of human behavior is a costly and time-consuming process, and hence the successful completion of an economic-gaming experiment requires the utmost in reliability of the time-sharing system. Even minor system fall-downs are usually intolerable to the experimental design; for they, at the very least, introduce possible lost data, i.e., lost observations on subject behavior or time delays in the operation of the experiment. A system crash normally causes the experimenter to terminate the experiment and dismiss the subjects and can even force cancellation or modification of an entire sequence of experiments if the system fall-down occurred at a particularly crucial point in the experimental design. For these reasons, the existence of an on-site time-sharing system is crucial to providing reliable service to experimenters. Only through on-site installations can control be exercised over the reliability of the hardware and of the system software. (3) the necessity of an on-site installation, combined with the meagre finances of most researchers in the behavioral sciences, requires that concessions be made in the design of hardware and software to provide economical service. Historically, these concessions have been the development of a language tailored to the needs of those programming experimental-gaming situations and the development of a single-language time-sharing system for that purpose. Further, the cost of extremely reliable mass-storage devices has prohibited their use thus far. (4) In addition to meeting the above constraints of fast time-shared operation on a small computing system, the language utilized by the system must (a) have provision for the usual computational requirements of a behavioral-science experiment. For example, the experimental program normally modeled in gaming situations requires that the language have facilities for matrix manipulations and elementary string operations. (b) The language must be relatively easy for novice programmers to learn and use; that is, behavioral scientists with little or no background in the computing sciences must readily comprehend the language. (c) The program should be self documenting, i.e., the language in which the model is programmed must be general enough so that the code is virtually machine independent. (d) The language must allow a limited real-time report of subject performance to the experimenter. The experimenter must be able to sample a subject's performance while the experiment is in progress; further, he must do so without the subject's recognizing that his performance is being monitored. (e) The language must enable the experimenter easily to exercise control over segments of an in-progress experiment. Very frequently, in the course of an experiment, the need arises for the experimenter to modify the nature of the experiment itself or to communicate with the subject by sending messages to him. This requirement and the previous one translate into the necessity of allowing a controlled amount of interaction between the time-shared terminals used by the subjects and the terminal used by the experimenter. (f) The language must permit a controlled amount of subject-to-subject interaction for bargaining and other small-group experiments. Again, this translates into a need for some degree of interaction among the users of the time-sharing system. (g) The system must store all relevant information on subject behavior in machine-readable form for subsequent data analysis. Data on subject behavior is usually analyzed statistically at the conclusion of the experiment, often on another com-

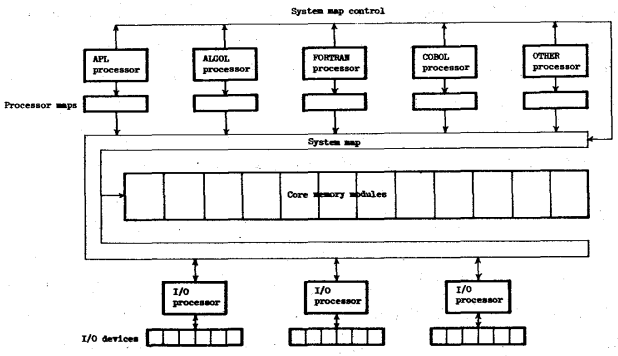


Figure 8—A time-shared array of language processors

From the collection of the Computer History Museum (www.computerhistory.org)

puter and the need for any keypunching is eliminated if all information can be recorded on a peripheral device in machine-readable form. (h) The language must interface the experiment to a variety of special-purpose input-output peripherals, such as galvanic skin response and other analogue equipment, video displays, sense switches and pulse relays for slide projectors, reward machines and the like. (i) A final requirement of the experimental-control language is the need for reentrancy. Reentrant coding permits the use of shared functions among users of the time-sharing system, thereby conserving core.

AN ARRAY OF LANGUAGE PROCESSORS (ALPS)

The Meta-APL Time-Sharing system which has been described represents a conceptual prototype for a time-shared array of language processors. (See Figure 8) The ALPS consist of an array of independent dedicated language processors which communicate with the outside world and/or each other exclusively through core memory. These processors are completely independent, and could indeed be different machines. The physical memory consists of core memory plus secondary storage and is divided in 4K blocks allocated to the various processors through the system map. These blocks appear to each processor through the system map as a continuous memory which is in turn paged via the processor maps.

Let us consider the successive transformations of an address issued by a language processor (Figure 9). The logical page number field of the address is used to access a location in the processor map whose contents represent the physical page number and are substituted in the page field of the address. The reconstituted address is then interpreted by the system map in a different way: an initial field of shorter length than the page number represents the virtual module number and is used to access a location within the system map which substitutes automatically its contents (physical module number) for the original field.

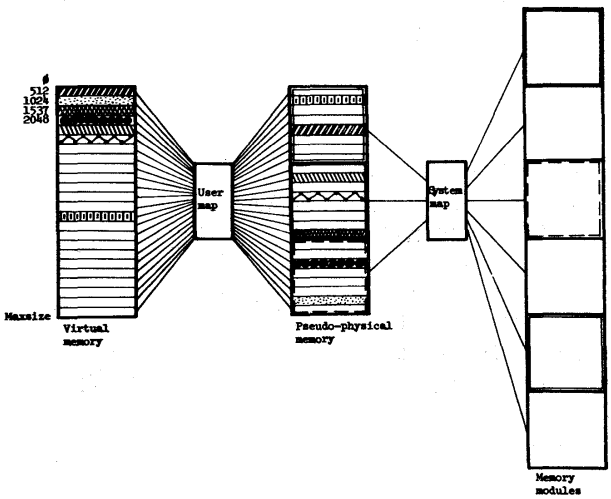


Figure 9—Memory mappings

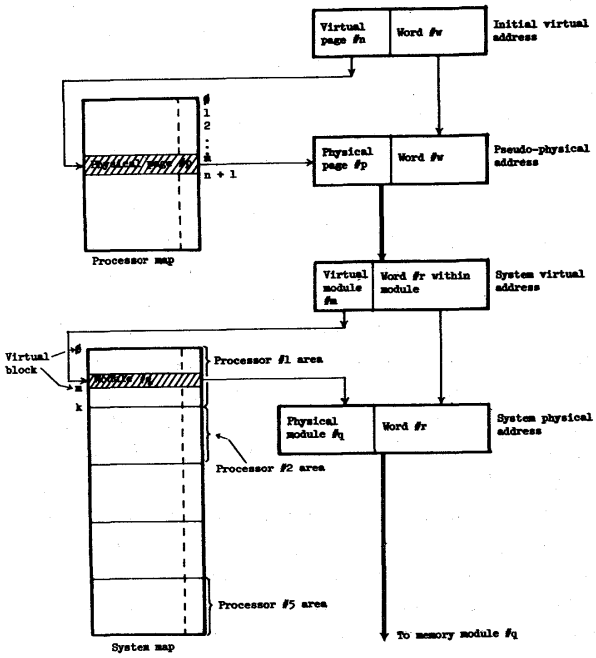


Figure 10—General address computation

This physical module number is then used to access a memory module while the rest of the address specifies a word within the module.

Note that if no processor is expected to monopolize all of core memory, there will be many more physical memory modules than virtual ones for each processor.

The physical module number field will then be much larger than the original virtual module number so that the size of physical memory which can be accessed by any one processor over a period of time can be much larger than its maximum addressing capability, as defined by the length of its instruction address field.

A user logging in on one of the ALPS terminals obtains the attention of the corresponding I/O processor and communicates with it via the system-wide command language. Once input has been completed a language processor is flagged by having the user's string

From the collection of the Computer History Museum (www.computerhistory.org)

assigned to its portion of the system map. Switching between languages is handled as a transfer within the system's virtual memory and therefore implemented as a mere system map modification. For this reason, all map handling routines are common to all processors, including I/O processors. Map protection is provided by hardware lock-out.

Finally, the modularity of the system provides a high degree of reliability. Core modules can be added or removed by merely marking their locations within the system map as full or empty. The same holds for the languages processors; to each of them corresponds a system map block containing one word per core module that may be allocated to it up to the maximum size of the processor's storage compatible with its addressing capabilities. Furthermore, each language processor say #n-1 might have access to an interpreter for language n written in language n-1 (mod the number of language processors), so that, should processor n be removed, processor n-1 could still interpret language n, the penalty being in this case a lower speed in execution.

Similarly, the number of I/O processors can be adjusted to the needs for input-output choices, terminals, or secondary storage choices.

It shou'd be noted that although the system is asynchronous and modular, the modules, memory as well as processors need not be identical. In fact, it seems highly desirable to use different processor architectures to interpret the various languages.

In summary, the essential features of the ALPS time-sharing system are:

—automatic address translation via a multilevel hardware mapping system; each user and each processor operates in its own virtual storage.
—the type, architecture, and characteristics of each processor are optimized for the computer language that it interprets, allowing for maximum technological efficiency for the language considered.

This is essentially a low-cost system since each processor has to worry about a single language, and the overhead for language swapping is reduced to merely switching the user to a different processor, allowing a smaller low-cost processor to operate efficiently in this environment.

ACKNOWLEDGMENTS

The authors are indebted to Messrs. A. C. Hoggatt and F. E. Balderston, who encouraged the development of the APL system. Appreciation is due to Mr. M. B. Garman for editorial suggestions and stimulating discussions, to Messrs. E. A. Stohr and Simcha Sadan for their work on APL operators, and to Mr. R. Rawson for his effort on the temporary IO processor. We are also grateful to Mrs. I. Workman for her very careful drawings and typing.

APPENDIX—SIMPLIFIED BNF META-APL EXTERNL SYNTAX

Notes: (1) {...} denotes ∅ or 1 times .... **( )** are symbols of Meta-APL.
(2) Lower-case letters are used for comments to avoid lengthy repetitions.
(3) cr denotes a carriage-return.
(4) PROGRAM in BNF is equivalent to PROCESS in the text.

Group 1

⟨FUNCTION BLOCK⟩ ::= ⟨FUNCTION DEFINITION⟩ ⟨STATEMENTS⟩ ∇
⟨PROGRAM BLOCK⟩ ::= {⟨PROGRAM DEFINITION⟩} ⟨PROGRAM⟩
⟨PROGRAM DEFINITION⟩ ::= ⟨PROGRAM NAME⟩ {(⟨PARAMETER LIST⟩)}
⟨PROGRAM NAME⟩ ::= ⟨NAME⟩
⟨PROGRAM⟩ ::= ⟨STATEMENT⟩ | ⟨PROGRAM⟩ ⟨STATEMENT⟩
| ⟨PROGRAM⟩ ⟨FUNCTION BLOCK⟩
⟨STATEMENT⟩ ::= {⟨LABEL⟩} ⟨STATEMENT LINE⟩ cr
⟨STATEMENT LINE⟩ ::= ⟨BRANCH⟩ | ⟨SPECIFICATION⟩ | ⟨SYSTEM COMMAND⟩
| ⟨IMMEDIATE⟩
⟨SYSTEM COMMAND⟩ ::= ⟨see system commands⟩
⟨BRANCH⟩ ::= → ⟨EXPRESSION⟩
⟨IMMEDIATE⟩ ::= ⟨EXPRESSION⟩ cr | ⟨EXPRESSION⟩ ; ⟨IMMEDIATE⟩
⟨SPECIFICATION⟩ ::= ⟨VARIABLE⟩←⟨EXPRESSION⟩ cr | ⟨OUTPUT SYMBOL⟩
←⟨EXPRESSION⟩

From the collection of the Computer History Museum (www.computerhistory.org)

Group 2

⟨NUMBER⟩ ::= ⟨DECIMAL FORM⟩ | ⟨EXPONENTIAL FORM⟩
⟨DECIMAL FORM⟩ ::= {⟨INTEGER⟩} . {⟨INTEGER⟩} | ⟨INTEGER⟩
⟨INTEGER⟩ ::= ⟨DIGIT⟩ | ⟨INTEGER⟩ ⟨DIGIT⟩
⟨DIGIT⟩ ::= ∅ | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9
⟨EXPONENTIAL FORM⟩ ::= ⟨DECIMAL FORM⟩ E ⟨INTEGER⟩
⟨VECTOR⟩ ::= ⟨SCALAR VECTOR⟩ | ⟨CHARACTER VECTOR⟩ | ⟨EMPTY VECTOR⟩
⟨SCALAR VECTOR⟩ ::= ⟨NUMBER⟩ | ⟨SCALAR VECTOR⟩ ⟨SPACES⟩ ⟨NUMBER⟩
⟨SPACES⟩ ::= ⟨SPACE⟩ | ⟨SPACES⟩ ⟨SPACE⟩
⟨SPACE⟩ ::= ⟨one blank⟩
⟨EMPTY VECTOR⟩ ::= '' | ⍳∅ | ⍴ ⟨SCALAR⟩
⟨CHARACTER VECTOR⟩ ::= '⟨CHARACTER STRING⟩'
⟨CHARACTER STRING⟩ ::= ⟨CHARACTER⟩ | ⟨CHARACTER STRING⟩ ⟨CHARACTER⟩
⟨CHARACTER⟩ ::= ⟨LETTER⟩ | ⟨DIGIT⟩ | ⟨SYMBOL⟩
⟨NAME⟩ ::= ⟨LETTER⟩ | ⟨LETTER⟩ ⟨ALPHANUMERIC STRING⟩
⟨ALPHANUMERIC STRING⟩ ::= ⟨ALPHANUMERIC⟩ | ⟨ALPHANUMERIC STRING⟩ ⟨ALPHANUMERIC⟩
⟨ALPHANUMERIC⟩ ::= ⟨LETTER⟩ | ⟨DIGIT⟩
⟨NUMERICAL TYPE⟩ ::= ⟨NUMBER⟩ | ⟨VECTOR⟩
⟨LOGICAL⟩ ::= ∅ | 1
⟨LABEL⟩ ::= ⟨NAME⟩ :
⟨IOSYMBOL⟩ ::= ⟨INPUT SYMBOL⟩ | ⟨OUTPUT SYMBOL⟩
⟨OUTPUT SYMBOL⟩ ::= □ | ⟨DEVICE ID⟩
⟨INPUT SYMBOL⟩ ::= □ | quote-quad
⟨DEVICE ID⟩ ::= ⟨undefined as yet⟩

Group 3

⟨SCALAR OPERATOR⟩ ::= ⟨MONADIC SCALOP⟩ | ⟨DYADIC SCALOP⟩
⟨MONADIC OPERATOR⟩ ::= ⟨MONADIC SCALOP⟩ | ⟨MONADIC MIXEDOP⟩ | ⟨MONADIC EXTENDED SCALOP⟩
⟨DYADIC OPERATOR⟩ ::= ⟨DYADIC SCALOP⟩ | ⟨DYADIC MIXEDOP⟩ | ⟨DYADIC EXTENDED SCALOP⟩
⟨MONADIC SCALOP⟩ ::= + | − | × | ÷ | ⌈ | ⌊ | * | log | | | ! | ? | 0 | ∼
⟨DYADIC SCALOP⟩ ::= ⟨MONADIC SCALOP⟩ | ∧ | ∨ | nand | nor | < | ≤ | ≥ | $> | $≠
⟨EXTENDED SCALAR OPERATOR⟩ ::= ⟨MONADIC EXTENDED SCALOP⟩ | ⟨DYADIC EXTENDED SCALOP⟩
⟨MONADIC EXTENDED SCALOP⟩ ::= ⟨SCALAR OPERATOR⟩ / | ⟨SCALAR OPERATOR⟩ /[COORD]
⟨DYADIC EXTENDED SCALOP⟩ | ⟨DYADIC SCALOP⟩ . ⟨DYADIC SCALOP⟩ | 0. ⟨DYADIC SCALOP⟩
⟨COORD⟩ ::= 1 | 2
⟨MIXED OPERATOR⟩ ::= ⟨DYADIC MIXEDOP⟩ ⟨MONADIC MIXEDOP⟩
⟨MONADIC MIXEDOP⟩ ::= ⍴ | , | ∼ | ⍋ | transpose/grade-up/grade-down | ∇ | Δ
⟨DYADIC MIXEDOP⟩ ::= ⍴ | , | ⍳ | ⍋ | transpose | / | \ | ↑ | ↓ | ∈ | ⊥ | ⊤ |

From the collection of the Computer History Museum (www.computerhistory.org)

Group 4

⟨EXPRESSION⟩ ::= ⟨NUMERICAL TYPE⟩ | ⟨VARIABLE⟩ | ⟨INPUT SYMBOL⟩
| ⟨MONADIC EXPRESSION⟩ | ⟨DYADIC EXPRESSION⟩
| ⟨MONADIC EXPRESSION⟩ | ⟨∅-ARG FUNCTION⟩

⟨MONADIC EXPRESSION⟩ ::= ⟨MONADIC OPERATOR⟩ ⟨EXPRESSION⟩
| ⟨1-ARG FUNCTION⟩

⟨DYADIC EXPRESSION⟩ ::= ⟨EXPRESSION⟩ ⟨DYADIC OPERATOR⟩ ⟨EXPRESSION⟩
| ⟨ALPHANUMERIC STRING⟩ ⟨RELOP⟩
⟨ALPHANUMERIC STRING⟩
| ⟨LOGICAL⟩ ⟨RELOP⟩ ⟨LOGICAL⟩ | ⟨2-ARG FUNCTION⟩

⟨RELOP⟩ ::= < | ≤ | = | ≥ | > | ≠

⟨FUNCTION NAME⟩ ::= ⟨NAME⟩

⟨∅-ARG FUNCTION⟩ ::= ⟨FUNCTION NAME⟩ {(⟨PARAMETER LIST⟩)}

⟨1-ARG FUNCTION⟩ ::= ⟨FUNCTION NAME⟩ {(⟨PARAMETER LIST⟩)}
⟨EXPRESSION⟩

⟨2-ARG FUNCTION⟩ ::= ⟨EXPRESSION⟩ ⟨FUNCTION NAME⟩
{(⟨PARAMETER LIST⟩)} ⟨EXPRESSION⟩

⟨FUNCTION DEFINITION⟩ ::= ∇{⟨VARIABLE NAME⟩}←{⟨VARIABLE NAME⟩}
⟨FUNCTION NAME⟩ {(⟨PARAMETER LIST⟩)}
⟨VARIABLE NAME⟩ {⟨LOCAL VARIABLES⟩}
| ∇{⟨VARIABLE NAME⟩}←⟨FUNCTION NAME⟩
{(⟨PARAMETER LIST⟩)} {⟨VARIABLE NAME⟩}
{⟨LOCAL VARIABLES⟩}

⟨VARIABLE NAME⟩ ::= ⟨NAME⟩

⟨VARIABLE⟩ ::= ⟨VARIABLE NAME⟩ | ⟨INDEXED VARIABLE⟩

⟨INDEXED VARIABLE⟩ ::= ⟨NAME⟩ [⟨EXPRESSION⟩ {;⟨EXPRESSION⟩}]

⟨PARAMETER LIST⟩ ::= ⟨PARAMETER NAME⟩ | ⟨PARAMETER LIST⟩
,⟨PARAMETER NAME⟩

⟨LOCAL VARIABLES⟩ ::= ;⟨VARIABLE NAME⟩
| ⟨LOCAL VARIABLE⟩ ; ⟨VARIABLE NAME⟩

⟨PARAMETER NAME⟩ ::= ⟨VARIABLE NAME⟩

⟨LETTER⟩ ::= *A* | *B* | *C* | *D* | *E* | *F* | *G* | *H* | *I* | *J* | *K* | *L* | *M*
| *N* | *O* | *P* | *Q* | *R* | *S* | *T* | *U* | *V* | *W* | *X* | *Y* | *Z*

⟨SYMBOL⟩ ::=] | [| ← | → | + | × | / | \ | , | . | ¨ | ¯ | <
| > | ≠ | ≤ | ≥ | = |) | (| ∨ | ∧ | ∈ | : | ⊥ | ⊤
| ⍱ | ↓ | ↑ | ⍳ | ∼ | ○ | ? | ⌊ | ⌈ | − | * | ⍴ | ∪
| ∩ | ⍺ | ⊂ | ⊃ | ÷ | ⍵ | ⍞ | ∘ | ∇ | Δ | ′ | (|)

From the collection of the Computer History Museum (www.computerhistory.org)